

Decision Tree Learning

Aijun An

Department of Electrical Engineering and Computer Science
York University

Outline

- Basic concepts in decision tree learning
- How to learn a decision/classification tree
 - Basic decision tree learning algorithm
 - How to select best attribute
 - Pruning decision tree
 - How to deal with numeric attributes
 - Other issues in decision tree learning
- Regression tree learning
- Bagging and Boosting

Decision Tree Learning

- Objective of decision tree learning
 - Learn a decision tree from a set of labeled training data
 - The learned tree can be used to make predictions on new examples
- Two types of decision trees
 - Classification tree
 - Purpose is to classify new examples
 - Target attribute is categorical
 - Regression tree
 - Purpose is to predict a numeric value for new examples
 - Target attribute is numeric

Example of Training Data

Input/Condition/Independent attributes						Target/Output/Decision/Dependent/Class attribute
Day	Outlook	Temperature	Humidity	Wind	PlayTennis	
D1	Sunny	Hot	High	Weak	No	
D2	Sunny	Hot	High	Strong	No	
D3	Overcast	Hot	High	Weak	Yes	
D4	Rainy	Mild	High	Weak	Yes	
D5	Rainy	Cool	Normal	Weak	Yes	Class or
D6	Rainy	Cool	Normal	Strong	No	Class label
D7	Overcast	Cool	Normal	Strong	Yes	
D8	Sunny	Mild	High	Weak	No	
D9	Sunny	Cool	Normal	Weak	Yes	
D10	Rainy	Mild	Normal	Weak	Yes	
D11	Sunny	Mild	Normal	Strong	Yes	
D12	Overcast	Mild	High	Strong	Yes	
D13	Overcast	Hot	Normal	Weak	Yes	
D14	Rainy	Mild	High	Strong	No	

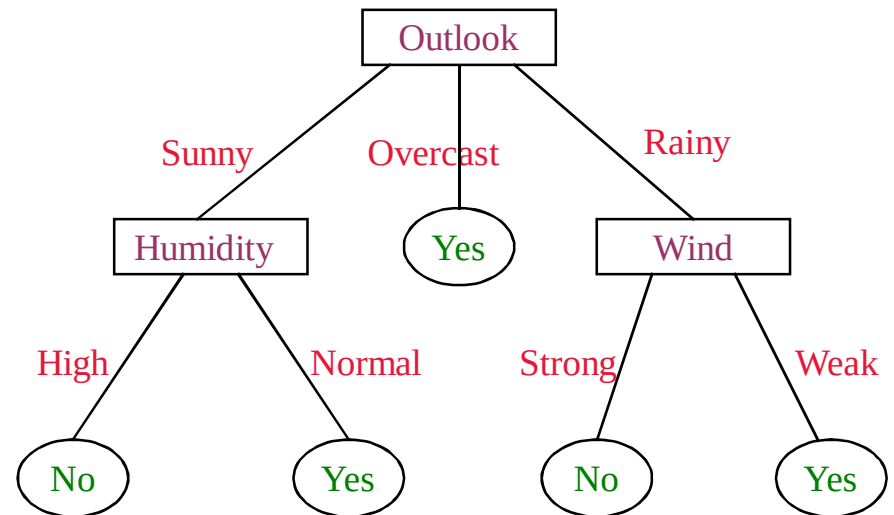
Training
Examples,
Instances,
Cases, or
Observations

for classification
problems

Class or
Class label

Decision Tree Representation

- *Decision tree*: a tree model for making decisions (e.g., classification)
 - Each non-leaf node tests a condition attribute (Outlook, Humidity, Wind)
 - Each branch corresponds to an attribute value
 - Each leaf node assigns a decision
 - a class for classification
 - a numeric value for regression
- To make a decision on a new example:
 - Test the example against the tree nodes from the root to a leaf node
 - For example:
⟨Outlook = Sunny, Temperature = Mild, Humidity = high, Wind = Strong⟩ ⇒ No



This tree classifies days according to whether or not they are suitable for playing tennis.

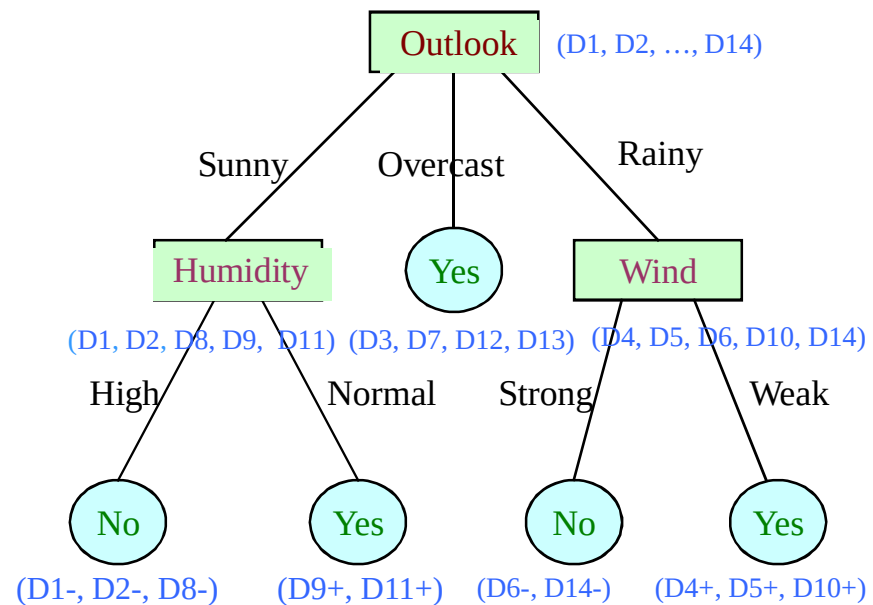
Decision Tree Learning Algorithms

- Many decision tree learning algorithms
 - ID3 (Quinlan, 1986)
 - C4.5 (Quinlan, 1993)
 - CART (Breiman, Friedman, et. al. 1983)
 - CHAID (Kass, 1980)
 - QUEST (Loh and Shih, 1997)
 - etc.
- They use the same basic tree learning algorithm:
 - Top-down induction of a decision tree

Top-Down Induction of a Decision Tree

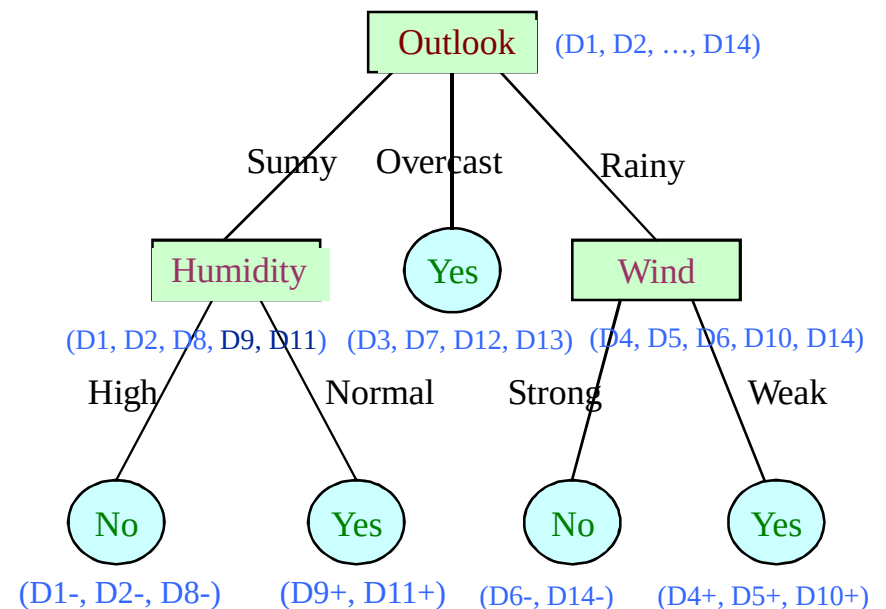
- ▶ Idea: divide and conquer
- ▶ Method: recursive partitioning of training data according to the attribute selected for a tree node.

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rainy	Mild	High	Weak	Yes
D5	Rainy	Cool	Normal	Weak	Yes
D6	Rainy	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rainy	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rainy	Mild	High	Strong	No



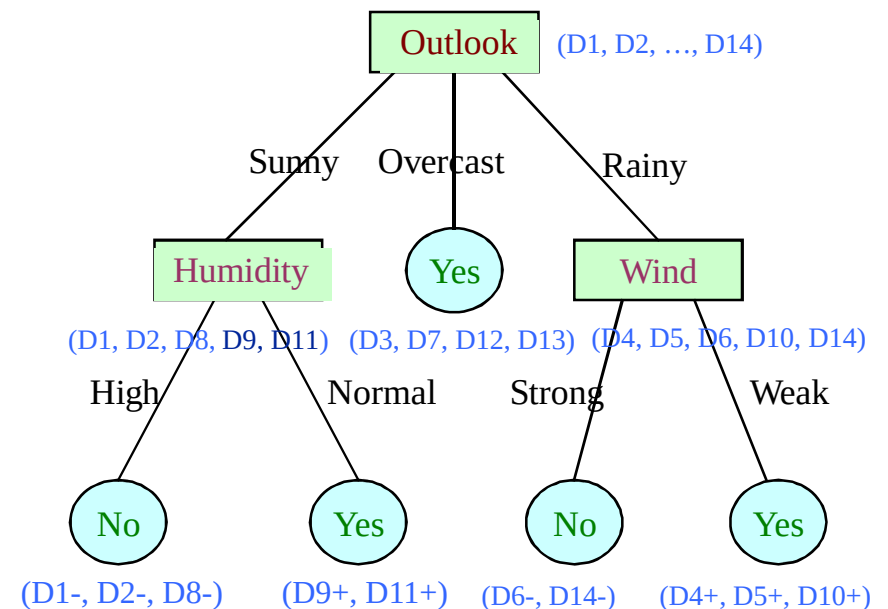
Issues in Basic Decision Tree Induction

- ▶ Three issues:
 - ▶ How to select an attribute for a node
 - ▶ When to declare a node terminal
 - ▶ How to assign a label to a leaf node
- ▶ Solutions depend on whether the tree is for classification or regression
- ▶ Solutions vary among algorithms



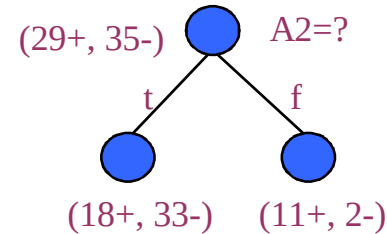
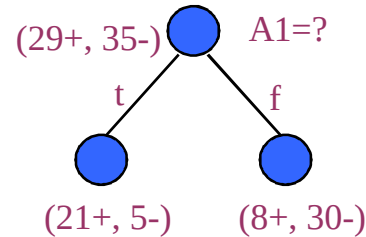
Solutions to the Issues in **Classification Tree** Induction

- ▶ How to select an attribute for a node
 - ▶ Use a criterion (to be described)
- ▶ When to declare a node terminal
 - ▶ A naïve, but not robust method:
when node is pure, stop growing.
 - ▶ Use a purity or another threshold
- ▶ How to assign a class to a leaf node
 - ▶ Assign the most common class in the examples in the node to the node
 - ▶ Or output the probability for each class



How to Select Attribute for Classification

- ▶ Which attribute is the best attribute given a set of attributes and a set of examples?



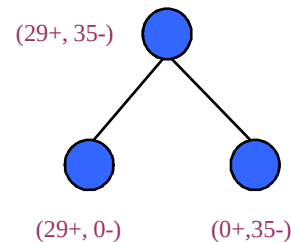
- ▶ Many selection criteria, including:
 - ▶ Information gain (Quinlan, 1983; used in ID3)
 - ▶ Gain ratio (Quinlan, 1986; used in C4.5)
 - ▶ Gini index (Breiman, 1984; used in CART)
 - ▶ Chi-square statistic (Kass, 1980; used in CHAID. Mingers, 1989)
 - ▶ Binarization (Bratko & Kononenko, 86)
 - ▶ Normalized information gain (Lopez de Mantaras, 91)

Information Gain

- ▶ Objective:

- ▶ Select an attribute so that the data in each of the descendant subsets are the “purest”.

An ideal split:



However, there may not be an attribute in the data set leading to such a split.

- ▶ To measure impurity, use *entropy*

- ▶ *Entropy* is a measure, commonly used in information theory, that characterizes the impurity (uncertainty, chaos) of an arbitrary collection of examples.

Entropy

- ▶ Given a set S of examples and k classes ($C_1, \dots, C_k$), the *entropy* of S with respect to the k classes is defined as:

$$Entropy(S) = -\sum_{i=1}^k P(C_i) \log_2(P(C_i))$$

where $P(C_i)$ is the probability of examples in S that belong to C_i .

- ▶ The bigger $Entropy(S)$ is, the more impure S is.
- ▶ Examples:
 - ▶ If all examples in S belong to the same class (i.e., S is pure),
 $Entropy(S)=0$
 - ▶ If half of the examples in S belong to class 1 and the other half belong to class 2,
 $Entropy(S)=1$
 - ▶ Suppose 9 examples are in class 1 and 5 examples in class 2,
 $Entropy(S)= -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) = 0.940$
 - ▶ If the examples are uniformly distributed in 3 classes,
 $Entropy(S)= - ((1/3)\log_2(1/3))\times 3 = \log_2 3 = 1.59$

Information Gain (*Cont'd*)

- ▶ An attribute-selection criterion:
 - ▶ Used to choose an attribute to split a data set
- ▶ Assume that attribute A has m values.
 - ▶ Using A , data set S is split into $S_1, S_2, \dots, S_m$.
- ▶ Information Gain

$Gain(S, A)$ = expected reduction in entropy due to partitioning S on attribute A

$$Gain(S, A) = Entropy(S) - \sum_{i=1}^m \frac{|S_i|}{|S|} Entropy(S_i)$$

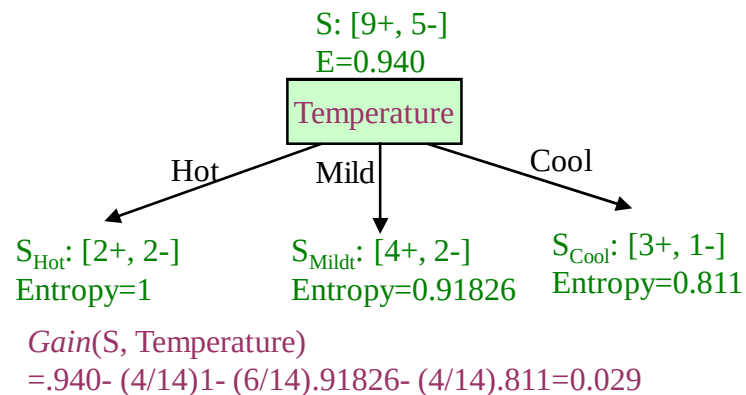
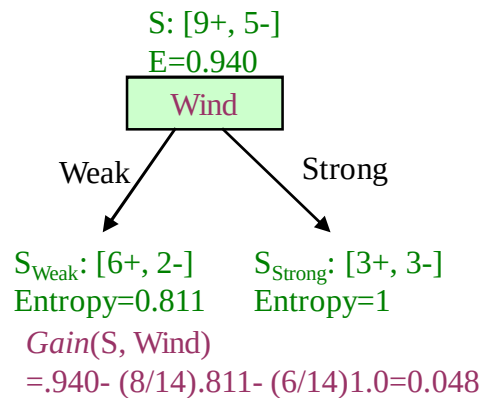
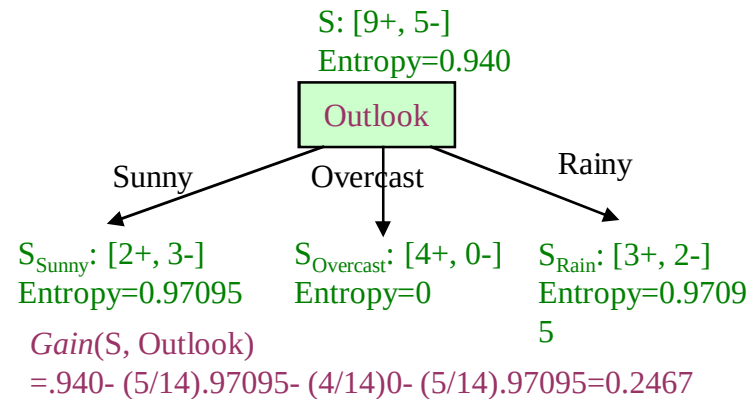
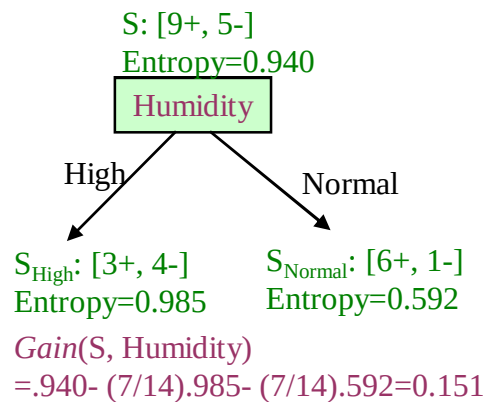
where $|S|$ is the number of examples in set S , and $|S_i|$ is the number of examples in S_i .

An illustrative example

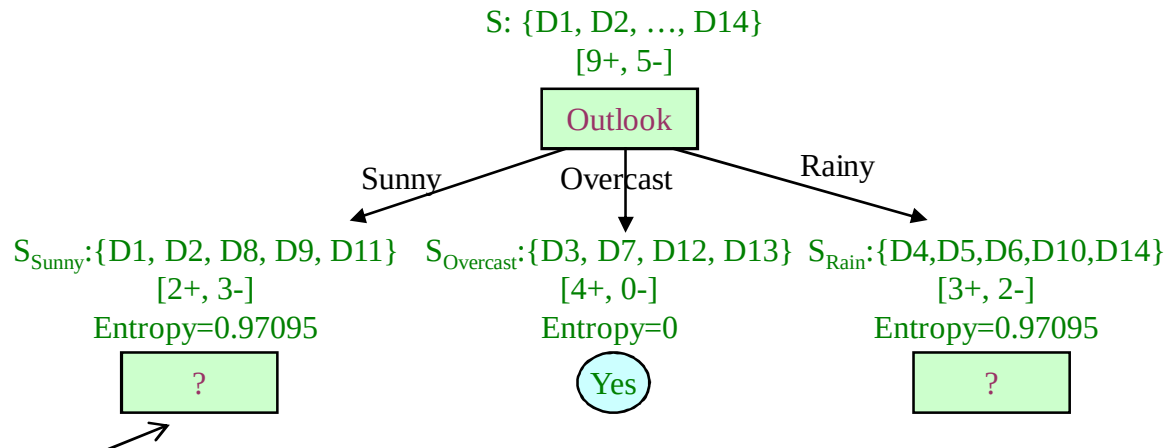
► Training examples

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rainy	Mild	High	Weak	Yes
D5	Rainy	Cool	Normal	Weak	Yes
D6	Rainy	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rainy	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rainy	Mild	High	Strong	No

Which attribute is the best for the root?



An illustrative example (Cont'd.)



Which attribute should be tested here, Humidity, Temperature, or Wind?

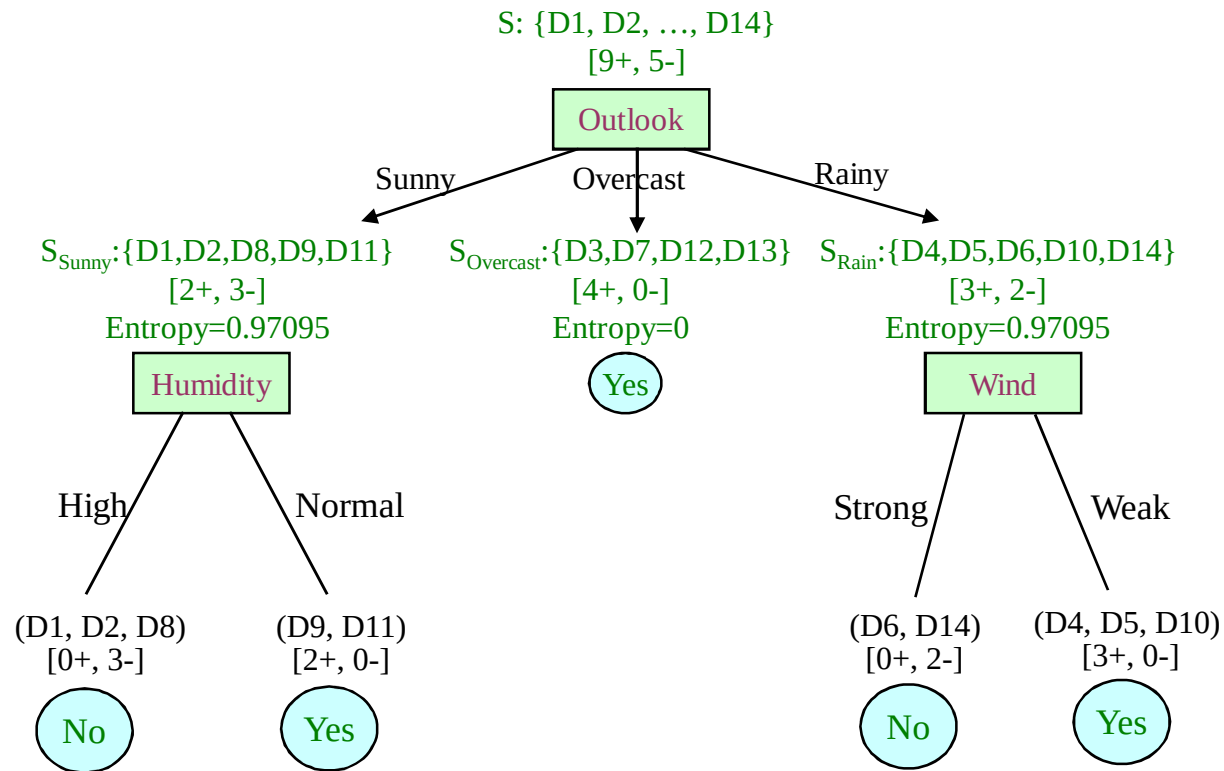
$$Gain(S_{\text{sunny}}, \text{Humidity}) = .97095 - (3/5)0.0 - (2/5)0.0 = 0.97095$$

$$Gain(S_{\text{sunny}}, \text{Temperature}) = .97095 - (2/5)0.0 - (2/5)1.0 - (1/5)0.0 = 0.57095$$

$$Gain(S_{\text{sunny}}, \text{Wind}) = .97095 - (2/5)1.0 - (3/5).918 = 0.02015$$

Therefore, **Humidity** is chosen as the next test attribute for the left branch.

An illustrative example (Cont'd.)



Basic Decision Tree Learning Algorithm

1. Select the “best” attribute A for the root node
2. Create new descendents of the node according to the values of A :
3. Sort training examples to the descendent nodes.
4. For each descendent node,
 - ▶ if there is no example for the node, the node is marked as a leaf node and labeled with the majority class in its parent node.
 - ▶ else if the training examples associated with the node belong to the same class, the node is marked as a leaf node and labeled with the class
 - ▶ else if there are no remaining attributes on which the examples can be further partitioned, the node is marked as a leaf node and labeled with the most common class among the training cases for classification;
 - ▶ otherwise, recursively apply the process on the new node.

*when to
terminate
the
recursive
process*

Outline

- Basic concepts in decision tree learning
- How to learn a decision tree
 - Basic decision tree learning algorithm
 - **How to select the best attribute**
 - Information gain ($\sqrt{}$)
 - Gain ratio
 - Gini index
 - Pruning decision tree
 - How to deal with numeric attributes
- Regression tree learning
- Bagging and Boosting

Bias in the Information Gain Measure

- ▶ Favor unfairly attributes with large numbers of distinct values at the expense of those with few.
 - ▶ E.g., attribute Date: poor predictor, but has the highest gain because it alone perfectly predicts the target attribute over the training data

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rainy	Mild	High	Weak	Yes
D5	Rainy	Cool	Normal	Weak	Yes
D6	Rainy	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rainy	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rainy	Mild	High	Strong	No

Gain Ratio

- ▶ Proposed by Quinlan in 1986 (used in C4.5)
- ▶ Idea:
 - ▶ penalizes attributes with many distinct values by dividing information gain by attribute information (**entropy of data with respect to the values of attribute**):

$$SplitInformation(S, A) = - \sum_{v_i \in Values(A)} \frac{|S_{v_i}|}{|S|} \log_2 \frac{|S_{v_i}|}{|S|} = - \sum_{v_i \in Values(A)} P(v_i) \log_2 P(v_i)$$

$$GainRatio(S, A) = \frac{Gain(S, A)}{SplitInformation(S, A)}$$

Gini Index

- ▶ Gini diversity index (used by CART)

- ▶ Another measure that measures the impurity of a data set.
- ▶ S is a set of training examples associated with a node
- ▶ Suppose there are n classes: C_i ($i = 1, \dots, n$)
- ▶ $P(C_i)$ is the probability of examples in S that belong to C_i .
- ▶ The Gini impurity of S with respect to classes can be measured as:

$$i(S) = \sum_{j \neq i} P(C_j)P(C_i) = 1 - \sum_{j=1}^n (P(C_j))^2$$

- ▶ Similar to entropy

- ▶ Minimized if classes for all examples are the same
 - ▶ Maximized if equal proportion of classes
- ▶ Selection of attribute using Gini index selects an attribute A that most reduces the impurity due to partitioning on A :

$$\Delta i(S, A) = i(S) - \sum_{v_i \in \text{Values}(A)} \frac{|S_{v_i}|}{|S|} i(S_{v_i})$$

Outline

- Basic concepts in decision tree learning
- How to learn a decision tree
 - Basic decision tree learning algorithm
 - How to select best attribute
 - *Pruning decision tree*
 - How to deal with numeric attributes
 - Other issues in decision tree learning
- Regression tree learning
- Bagging and Boosting

Basic Decision Tree Learning Algorithm

1. Select the “best” attribute A for the root node
2. Create new descendents of the node according to the values of A :
3. Sort training examples to the descendent nodes.
4. For each descendent node,
 - ▶ if the training examples associated with the node belong to the same class, the node is marked as a leaf node and labeled with the class
 - ▶ else if there are no remaining attributes on which the examples can be further partitioned, the node is marked as a leaf node and labeled with the most common class among the training cases for classification;
 - ▶ else if there is no example for the node, the node is marked as a leaf node and labeled with the majority class in its parent node.
 - ▶ otherwise, recursively apply the process on the new node.

*when to
terminate
the
recursive
process*

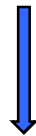
Overfitting Problem

- ▶ When to declare a node terminal in the basic algorithm:
 - ▶ grow each branch of the tree just deeply enough to classify the training examples as perfectly as possible.
- ▶ This strategy may lead to producing deep branches that cover very few examples.
- ▶ This kind of trees *overfits* the training data in the following two situations:
 - ▶ there is noise in the data → tree fits the noise.
 - ▶ the number of training examples is too small to produce a representative sample of the true target function → tree is too specific to classify future examples well.

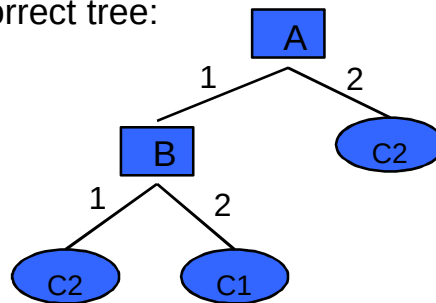
Example of Overfitting Problem

A	B	C	Class
1	1	1	C2
1	2	1	C1
2	1	2	C2
1	2	1	C1
1	2	2	C2
1	2	1	C1
2	1	1	C2
1	2	1	C1

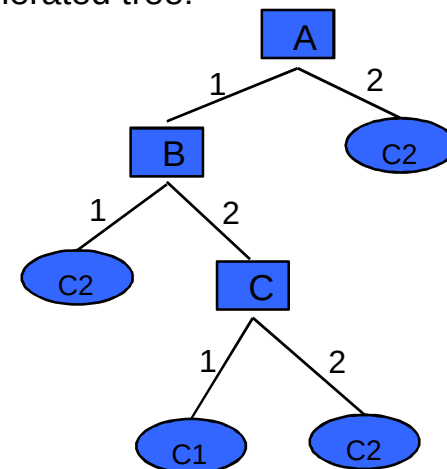
- Assume the model is:
If $A=1 \wedge B=2$, Class=C1;
Otherwise, Class=C2



Correct tree:



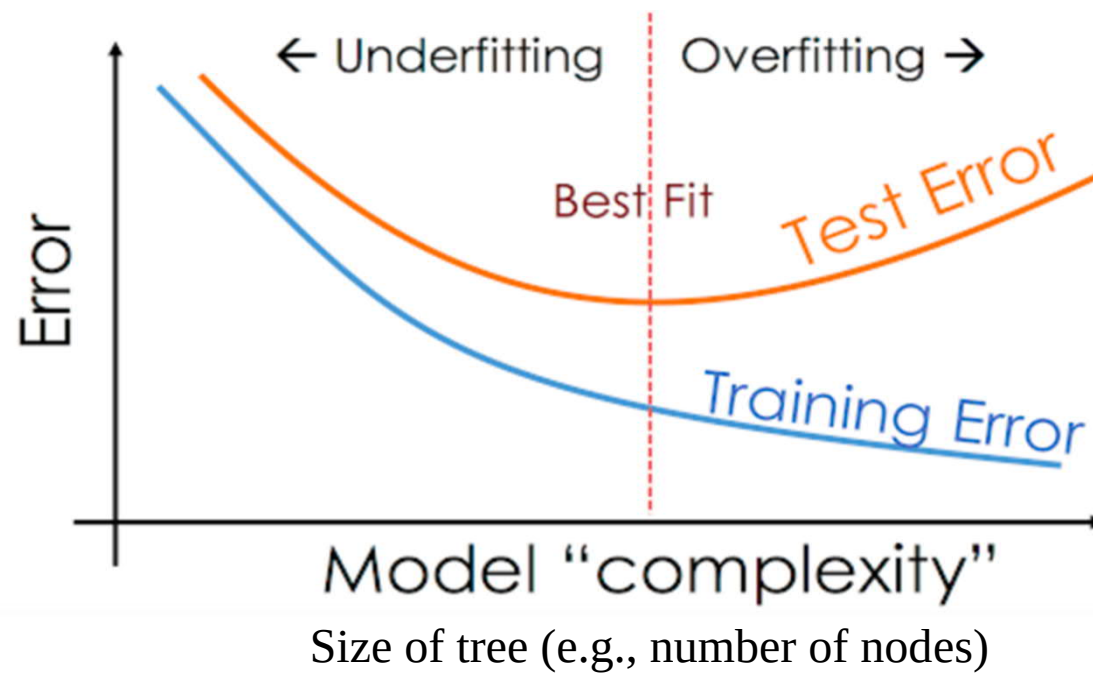
Generated tree:



Overfitting Problem (*Cont'd*)

- ▶ A definition of *overfitting*
 - ▶ Consider the error of a model (e.g., a tree) h over
 - ▶ training data: $error_{train}(h)$
 - ▶ Percentage of examples in training data misclassified by h
 - ▶ entire distribution D of data: $error_D(h)$
 - ▶ Percentage of examples in entire data misclassified by h
 - ▶ Model h overfits training data if there is an alternative model h' such that
$$error_{train}(h) < error_{train}(h') \text{ and } error_D(h) > error_D(h')$$

Overfitting in Decision Tree Learning (An Example Diagram)



Occam's Razor

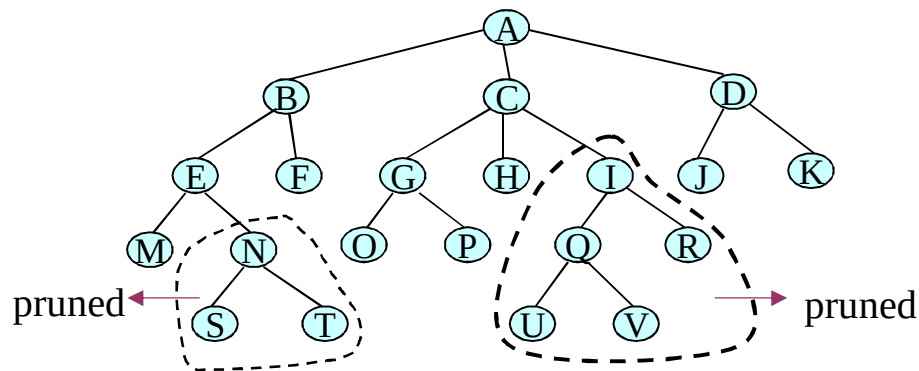
- Prefer short hypotheses that fit the observations
- Argument in favor:
 - Fewer short hypotheses than long ones
 - A short hypothesis that fits data unlikely to be a coincidence
 - A long hypothesis that fits data might be a coincidence
- Argument opposed:
 - There are many ways to define short hypotheses
 - What's so special about small sets based on the size of hypothesis
- Solution:
 - Find best/good short hypotheses (not random short ones)
 - Remove unnecessary parts of hypotheses

Preventing Overfitting

- ▶ **Pre-pruning:** stop growing the tree when data split is not statistically significant
 - ▶ For example,
 - ▶ set a threshold $\alpha > 0$ and declare a node terminal if percentage of examples in the most common class $> \alpha$, or
 - ▶ set a threshold $\beta > 0$ and declare a node terminal if highest information gain $< \beta$
 - ▶ Problems:
 - ▶ Hard to set the threshold value
 - ▶ The splitting is either stopped too soon or continued too far depending on the threshold
 - ▶ Using more complicated stopping rule does not help

Preventing Overfitting (Cont'd)

- ▶ **Post-pruning:** grow full tree, allowing it to overfit the data, and then remove some subtrees



- ▶ More successful in practice
- ▶ Criterion is needed to determine what to prune

Post-pruning Functions

- ▶ Post-pruning functions are needed to determine which part(s) of the tree should be pruned
- ▶ Several post-pruning functions, including:
 - ▶ Reduced-error (Quinlan, 83)
 - ▶ Error-complexity (CART, 84)
 - ▶ Pessimistic error (Quinlan, 86)
 - ▶ Minimum description length (MDL) (SLIQ by Mehta *et al.* 1996)
 - ▶ Minimum error (Niblett & Bratko, 86)
 - ▶ Critical value (Mingers, 87)
- ▶ There is no single best pruning algorithm

Reduced-Error Pruning

- ▶ Procedure
 - ▶ Split *training* data into *building* and *validation* sets
 - ▶ Generate an overfitting decision tree using the *building* set
 - ▶ Post-pruning: Do until further pruning is harmful:
 - ▶ Consider each of the internal non-root nodes in the tree to be candidates for pruning
 - ▶ Prune a node by removing subtree rooted at this node, making it a leaf node, and assigning it the most common classification of the training examples affiliated with this node
 - ▶ Evaluate impact of pruning this node on the *validation* set by
 - ▶ calculating the classification error rate of the pruned tree on the *validation* set and comparing it with the error rate of the unpruned tree.
 - ▶ Greedily remove the one whose removal most reduces the error on *validation* set.
- ▶ Aim to produce smallest version of most accurate subtree
- ▶ Limitations:
 - ▶ May be sub-optimal
 - ▶ When data is limited, data used to build the tree is small

Error-Complexity Pruning

- ▶ Similar procedure to the reduced-error pruning
 - ▶ Split training data into *building* and *validation* sets
 - ▶ Generate the decision tree with the *building* set
 - ▶ Post-pruning: Do until further pruning is harmful:
 - ▶ Evaluate impact of pruning each subtree on the *validation* set
 - ▶ Greedily remove the one whose removal (resulting in tree T) minimizes the following expression on the *validation* set:

$$R_{\alpha}(T) = E(T) + \alpha L(T)$$

where $E(T)$ is the classification error of tree T , $L(T)$ is the number of leaf nodes in T and α is the **complexity** cost per leaf node.

- ▶ $R_{\alpha}(T)$ is a linear combination of the classification error rate of the tree and its complexity.
- ▶ Similar to reduced-error pruning,
 - ▶ not suitable if the size of training data is small.
 - ▶ sub-optimal

Pessimistic-Error Pruning

- ▶ This method does not require a separate validation set.
- ▶ Suppose a subtree T_s contains L leaves and J of training examples associated with T_s are misclassified by the tree
- ▶ If we replace T_s with a leaf which misclassifies E of the associated training examples, the pruned tree will be accepted if

$$E + 1/2 < J + L/2$$

- ▶ Perform top-down traversing over internal nodes
 - ▶ If an internal node is pruned, all its descendants are removed
 - ▶ Relatively fast pruning
- ▶ Advantage
 - ▶ fast: no need to separate growing and validation sets.
 - ▶ Tree building makes use of all the training data.